



Модуль 9. Интеграция ML в асинхронный бэкенд

Введение в модуль

К этому моменту мы построили надёжный асинхронный бэкенд: он валидирует запросы, управляет соединениями с базой данных, аутентифицирует пользователей и масштабируется горизонтально. Но центральная задача ML-сервиса — не HTTP, а **инференс**: пропускание данных через обученную модель и получение предсказания.

Инференс — это особая нагрузка. Он может занимать миллисекунды (логистическая регрессия на CPU) или секунды (диффузионная модель на GPU). Он может требовать гигабайтов RAM или специализированного железа (TPU, GPU с Tensor Cores). Он может быть детерминированным или стохастическим (генеративные модели выдают разный текст при каждом запуске).

В этом модуле мы разберём:

- как выбрать архитектуру инференса под конкретную latency и пропускную способность;
- какие форматы моделей и серверы существуют и почему ONNX Runtime может быть в 10 раз быстрее чистого PyTorch;
- как не блокировать event loop тяжёлым вычислением и не терять задачи при перезапуске сервера;
- как организовать потоковую выдачу результатов (token-by-token) и real-time обработку аудио/видео;
- откуда брать фичи для модели и почему разделение на offline/online фичи критично для latency.

9.1. Архитектурные паттерны ML-инференса

9.1.1. Синхронный inference в endpoint

Самый простой паттерн: запрос пришёл, модель предсказала, ответ

ушёл — всё в рамках одного HTTP-запроса.

```
In [ ]: @app.post("/predict")
        async def predict(input_data: InputModel):
            features = preprocess(input_data)
            result = model.predict(features) # <- блокирует, если не осторожен
            return {"prediction": result.tolist()}
```

Когда это допустимо:

- Модель лёгкая: время inference < 100 мс.
- Модель не требует GPU (scikit-learn, небольшая нейросеть на CPU).
- Нагрузка предсказуема, пиков нет.

Проблема: если `model.predict` — синхронный вызов (например, PyTorch в CPU-режиме или NumPy-операции), он **блокирует event loop**.

Все остальные запросы встают. Мы обсуждали это в Модуле 2:

`time.sleep(10)` и тяжёлые вычисления — смерть для асинхронности.

Решение для лёгких моделей: запуск в `ProcessPoolExecutor`:

```
In [ ]: from concurrent.futures import ProcessPoolExecutor
        import asyncio

        executor = ProcessPoolExecutor(max_workers=4)

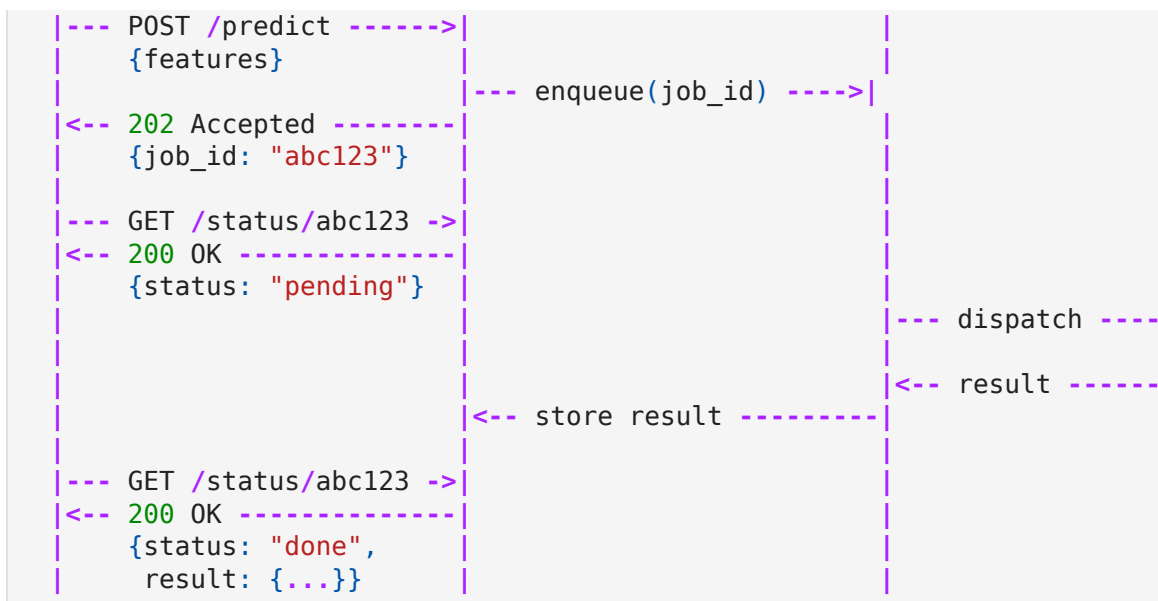
        @app.post("/predict")
        async def predict(input_data: InputModel):
            features = preprocess(input_data)
            loop = asyncio.get_running_loop()
            result = await loop.run_in_executor(executor, model.predict, features)
            return {"prediction": result.tolist()}
```

Но `run_in_executor` имеет накладные расходы: сериализация `pickle` для аргументов и результата. Для больших тензоров (изображения 4K, батчи из 100 примеров) это может занимать десятки миллисекунд.

9.1.2. Асинхронный inference: очереди задач

Если inference тяжёлый (> 500 мс) или непредсказуемый (генеративные модели), держать HTTP-соединение открытым всё это время — расточительство. Паттерн «запрос-ответ» превращается в «запрос — job ID — опрос статуса».

```
In [ ]: Клиент                                FastAPI                                Очередь
         |                                     |                                     |
```



Преимущества:

- Клиент не ждёт с открытым TCP-соединением.
- Можно масштабировать worker'ы независимо от FastAPI-серверов.
- Можно приоритизировать задачи (VIP-клиенты, real-time vs batch).
- Если worker упал — задача остаётся в очереди и будет переотправлена.

Недостатки:

- Добавляет сложность (брокер, worker'ы, хранилище результатов).
- Увеличивает latency (минимум накладные расходы очереди + время ожидания в очереди).

9.1.3. Batch inference: накопление и динамический батчинг

Модели на GPU эффективнее работают с **батчами**: параллельная обработка N примеров на одном forward pass значительно быстрее, чем N отдельных проходов.

Статический батчинг: клиент сам отправляет массив из N примеров.

```
In [ ]: @app.post("/predict/batch")
        async def predict_batch(inputs: list[InputModel]):
            batch = np.stack([preprocess(x) for x in inputs])
            results = model.predict(batch)
            return {"predictions": results.tolist()}
```

Динамический батчинг (dynamic batching): сервер накапливает входящие запросы в течение короткого окна (например, 10 мс) или до достижения максимального размера батча, затем выполняет один forward pass для всех накопленных запросов.

```
In [ ]: Время: 0ms    5ms    10ms
Запросы: R1 -> R2 -> R3 -> [BATCH: R1+R2+R3] -> inference -> ответы ВС
```

Это реализуется в специализированных ML-серверах (Triton Inference Server, TorchServe). FastAPI сам по себе не делает динамический батчинг — для этого нужна либо внешняя очередь с агрегатором, либо интеграция с Triton.

9.1.4. Математическая подоплека: теория очередей и оптимальный батч

Модель M/M/1 для inference

Рассмотрим один worker (один GPU/CPU), обрабатывающий запросы по мере поступления. Запросы приходят с интенсивностью λ (RPS), время обслуживания — экспоненциально со средним $S = \frac{1}{\mu}$.

Коэффициент загрузки:

$$\rho = \frac{\lambda}{\mu} = \lambda \cdot S$$

Условие стабильности: $\rho < 1$.

Среднее время ожидания в очереди (закон Литтла):

$$W_q = \frac{\rho}{\mu(1-\rho)} = \frac{\lambda S^2}{1 - \lambda S}$$

Вывод: при $\rho \rightarrow 1$ время ожидания взрывается. Если inference занимает 1 секунду, а вы присылаете 0.99 запроса в секунду — среднее ожидание в очереди:

$$W_q = \frac{0.99 \cdot 1}{1 - 0.99} = 99 \text{ секунд}$$

Модель M/M/c: несколько worker'ов

Если есть c параллельных worker'ов (например, 4 процесса в ProcessPoolExecutor или 4 GPU):

$$\rho = \frac{\lambda}{c\mu}$$

Вероятность того, что запросу придётся ждать (все worker'ы заняты), по формуле Эрланга C:

$$P_{\text{wait}} = \frac{\frac{c}{\rho}}{\sum_{k=0}^{c-1} \frac{(c\rho)^k}{k!} + \frac{(c\rho)^c}{c!(1-\rho)}}$$

При $\rho = 0.5$ и $c = 4$: $P_{\text{wait}} \approx 0.08$ (8% запросов ждут).

При $\rho = 0.8$: $P_{\text{wait}} \approx 0.42$ (42% запросов ждут).

Практический вывод: держите $\rho < 0.7$ для приемлемой latency.

Оптимальный размер батча

Пусть $T(n)$ — время inference для батча размера n . Обычно:

$$T(n) = T_{\text{fixed}} + n \cdot T_{\text{per_item}}$$

где T_{fixed} — фиксированные накладные расходы (загрузка в GPU, инициализация ядер). Из-за T_{fixed} увеличение n снижает среднее время на один пример:

$$\text{Latency per item} = \frac{T(n)}{n} = \frac{T_{\text{fixed}}}{n} + T_{\text{per_item}}$$

Но увеличение n увеличивает **время ожидания формирования батча** (accumulation delay). Если запросы приходят с интервалом $\frac{1}{\lambda}$, то среднее время ожидания последнего запроса в батче:

$$W_{\text{batch}} = \frac{n-1}{2\lambda}$$

Оптимизация: минимизировать суммарную latency:

$$\text{Total}(n) = W_{\text{batch}} + T(n) = \frac{n-1}{2\lambda} + T_{\text{fixed}} + n \cdot T_{\text{per_item}}$$

Это линейная функция от n . Если T_{fixed} велико и λ высока — большой батч выгоден. Если λ низкая (редкие запросы) или latency критична — батчинг убивает производительность, лучше обрабатывать по одному.

9.2. ML-серверы и форматы моделей

9.2.1. ONNX Runtime, TensorRT, OpenVINO

ONNX (Open Neural Network Exchange) — открытый формат представления моделей. PyTorch, TensorFlow, scikit-learn, Hugging Face — все могут экспортировать в ONNX.

ONNX Runtime — движок inference, оптимизирующий граф модели:

- Удаляет мёртвый код.
- Сликает операции (conv + batch norm -> fused conv).
- Использует векторные инструкции CPU (AVX512) или GPU (CUDA, TensorRT execution provider).

```
In [ ]: import onnxruntime as ort

session = ort.InferenceSession("model.onnx", providers=["CUDAAExecutionPro
inputs = {session.get_inputs()[0].name: features}
result = session.run(None, inputs)
```

TensorRT — NVIDIA-специфичный оптимизатор. Преобразует модель в формат, максимально использующий Tensor Cores (mixed precision FP16/INT8). Даёт ускорение 2–10× по сравнению с PyTorch на GPU, но:

- Работает только на NVIDIA GPU.
- Процесс конвертации может занимать минуты (build engine).
- Не все операции поддерживаются.

OpenVINO — Intel-специфичный. Оптимизирует для CPU Intel (использует VNNI для INT8) и GPU Intel. Особенно эффективен для edge-устройств.

9.2.2. Специализированные ML-серверы

TorchServe

Официальный сервер от PyTorch. Поддерживает:

- Модели в формате MAR (Model Archive).
- REST и gRPC API.
- Batch inference.
- A/B testing моделей.
- Метрики Prometheus.

```
In [ ]: torch-model-archiver --model-name my_model --version 1.0 --model-file mod
torchserve --start --model-store model_store --models my_model=my_model.m
```

Triton Inference Server (NVIDIA)

Промышленный стандарт для GPU-инференса. Поддерживает:

- ONNX, TensorRT, PyTorch, TensorFlow, Python backend.
- **Dynamic batching**: автоматическое накопление запросов в батч.
- **Model ensemble**: конвейер из нескольких моделей (предобработка -> inference -> постобработка).
- **GPU sharing**: несколько моделей на одном GPU с разделением памяти.
- gRPC и HTTP/REST.

Конфигурация dynamic batching:

```
In [ ]: dynamic_batching {
    preferred_batch_size: [ 4, 8 ]
    max_queue_delay_microseconds: 100
}
```

Triton ждёт до 100 мкс, пытаясь собрать батч из 4 или 8 запросов. Если не успевает — отправляет то, что есть.

BentoML

Python-native фреймворк для упаковки и сервировки моделей.

Интегрируется с FastAPI-подобным синтаксисом:

```
In [ ]: import bentoml

@bentoml.service(resources={"gpu": 1})
class MyService:
    def __init__(self):
        self.model = bentoml.models.get("my_model:latest").load()

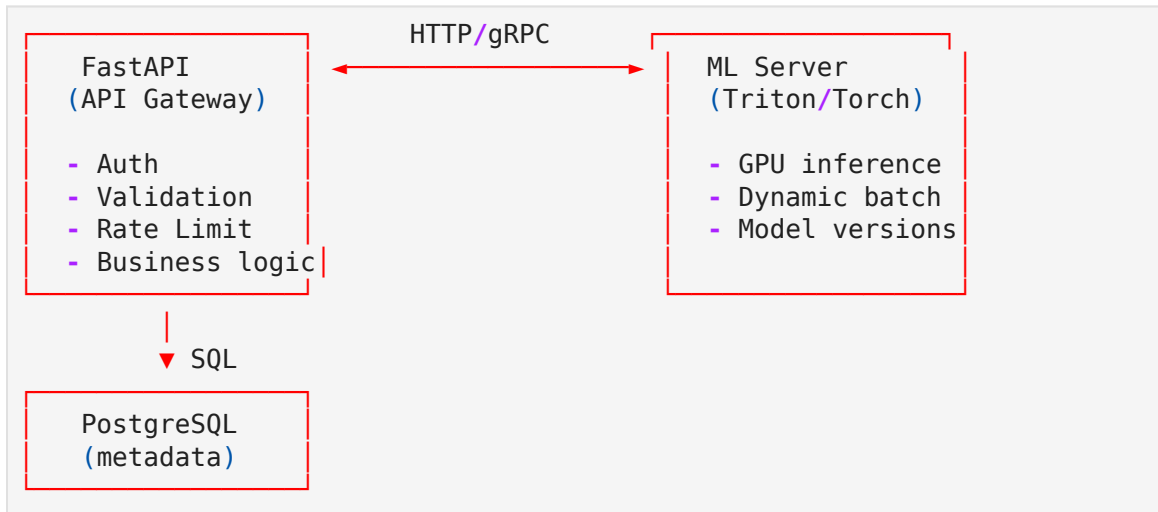
    @bentoml.api
    def predict(self, input_data: np.ndarray) -> np.ndarray:
        return self.model.predict(input_data)
```

BentoML автоматически генерирует Docker-образ, настраивает батчинг, adaptive traffic control.

9.2.3. Отделение ML-сервиса от API-сервиса

Рекомендуемая архитектура для production:

In []:



FastAPI не делает inference сам. Он:

1. Принимает запрос, валидирует, аутентифицирует.
2. Преобразует в формат, ожидаемый ML-сервером.
3. Отправляет в Triton/TorchServe по gRPC (быстрее HTTP для больших payload'ов).
4. Получает результат, обогащает метаданными, записывает в БД.
5. Возвращает клиенту.

Это позволяет:

- Масштабировать API и ML независимо.
- Обновлять модель без редеплоя API.
- Использовать разное железо (API на CPU, ML на GPU).

9.3. Фоновые задачи в FastAPI

9.3.1. BackgroundTasks : простые операции

FastAPI предоставляет лёгкий механизм для задач, которые не должны задерживать ответ клиента:

In []:

```
from fastapi import BackgroundTasks
```



```

async def send_notification(email: str, message: str):
    await email_service.send(email, message)

@app.post("/predict")
async def predict(
    input_data: InputModel,
    background_tasks: BackgroundTasks
):
    result = await model_service.predict(input_data)

    # Отправка уведомления в фоне, не ждём завершения
    background_tasks.add_task(send_notification, "user@example.com", "Don

    return {"prediction": result}

```

Как это работает:

- `add_task` регистрирует корутину в списке задач Starlette.
- После отправки HTTP-ответа клиенту Starlette последовательно выполняет все `BackgroundTasks`.
- Если задача падает с исключением — оно логируется, но не влияет на уже отправленный ответ.

Ограничения (критически важны):

- `BackgroundTasks` выполняются в **том же процессе**, что и endpoint.
- Если сервер перезапускается (деплой, падение) — невыполненные задачи **теряются**.
- Нет retry, нет persistence, нет мониторинга очереди.
- Не подходит для длительных задач (> нескольких секунд), так как блокирует graceful shutdown.

Когда использовать: логирование, отправка аналитики, email-уведомления — всё, что не критично и быстрое.

9.3.2. Celery: классика распределённых задач

Celery — зрелый фреймворк для фоновых задач с брокерами сообщений.

Архитектура:

In []: FastAPI → Broker (Redis/RabbitMQ) → Worker(s) → Result Backend (Red

Определение задачи:

In []: `from celery import Celery`

```

celery_app = Celery(
    "ml_tasks",
    broker="redis://localhost:6379/0",
    backend="redis://localhost:6379/0"
)

@celery_app.task(bind=True, max_retries=3)
def heavy_inference(self, model_id: str, features: list):
    try:
        model = load_model(model_id)
        result = model.predict(np.array(features))
        return {"status": "success", "result": result.tolist()}
    except Exception as exc:
        raise self.retry(exc=exc, countdown=60)

```

Запуск из FastAPI:

```

In [ ]: from fastapi import FastAPI

@app.post("/predict/async")
async def predict_async(input_data: InputModel):
    task = heavy_inference.delay("model_v1", input_data.features)
    return {"task_id": task.id, "status": "queued"}

@app.get("/predict/status/{task_id}")
async def get_status(task_id: str):
    task = celery_app.AsyncResult(task_id)
    return {
        "task_id": task_id,
        "status": task.status, # PENDING, SUCCESS, FAILURE, RETRY
        "result": task.result if task.ready() else None
    }

```

Преимущества Celery:

- Persistence: задача в Redis/RabbitMQ переживёт рестарт FastAPI.
- Retry с экспоненциальным backoff.
- Масштабирование: десятки worker'ов на разных машинах.
- Мониторинг через Flower (web-интерфейс).

Недостатки:

- Celery не нативно асинхронен (worker'ы — синхронные процессы).
- Накладные расходы на сериализацию (pickle/json).
- Сложность конфигурации (брокер, backend, worker'ы, beat для периодических задач).

9.3.3. Современные альтернативы: `arq`, `dramatiq`, `saq`

`arq` (async-native, на Redis)

Полностью асинхронный, построен на `asyncio`. Использует Redis Streams.

```
In [ ]: from arq import create_pool, Actor
        from arq.connections import RedisSettings

        async def predict_task(ctx, model_id: str, features: list):
            model = load_model(model_id)
            return model.predict(np.array(features)).tolist()

        class WorkerSettings:
            redis_settings = RedisSettings()
            functions = [predict_task]

        # B FastAPI
        redis = await create_pool(RedisSettings())

        @app.post("/predict/async")
        async def predict_async(input_data: InputModel):
            job = await redis.enqueue_job('predict_task', "model_v1", input_data)
            return {"job_id": job.job_id}
```

Преимущества `arq`:

- Нативная асинхронность: worker'ы — `async`, могут делать `await` без блокировок.
- Простота: нет необходимости в отдельном брокере (использует Redis).
- Type hints: задачи типизированы.

`dramatiq`

Простой, надёжный, с middleware (rate limiting, retries, Prometheus). Брокер — RabbitMQ или Redis. Синхронный, но легковеснее Celery.

`saq` (Simple Async Queue)

Минималистичный, `async`, на Redis. Поддерживает cron-задачи. Идеален, если нужен «Celery-lite» без легаси.

Выбор для ML-сервиса:

- **Celery**: если нужна зрелая экосистема, периодические задачи, сложные workflow (chains, chords).
- **arq**: если весь стек async (Python 3.11+), и нужна простота.
- **saq**: если нужен минимальный async worker с cron.

9.4. Потокковая обработка и WebSocket для ML

9.4.1. StreamingResponse для генеративных моделей

Генеративные модели (GPT, LLaMA, Stable Diffusion с intermediate steps) выдают результат не целиком, а по частям. Держать клиента в ожидании 30 секунд без обратной связи — плохой UX.

```
In [ ]: from fastapi import FastAPI
        from fastapi.responses import StreamingResponse
        import asyncio

        app = FastAPI()

        async def token_streamer(prompt: str):
            # Имитация генерации токенов
            tokens = ["The", " future", " of", " AI", " is", " bright", "."]
            for token in tokens:
                await asyncio.sleep(0.5) # имитация inference
                yield token

        @app.post("/generate")
        async def generate(prompt: str):
            return StreamingResponse(
                token_streamer(prompt),
                media_type="text/plain",
            )
```

Клиент получает данные по мере готовности:

```
In [ ]: HTTP/1.1 200 OK
        Content-Type: text/plain
        Transfer-Encoding: chunked

        The
         future
         of
```

AI

...

Важно для ML: реальная генерация токенов в LLM — это цикл, где каждый новый токен подаётся обратно на вход модели. StreamingResponse позволяет начать отправку первого токена сразу, не дожидаясь генерации всего текста.

9.4.2. Server-Sent Events (SSE)

SSE — стандартный механизм push-уведомлений поверх HTTP. Однонаправленный: сервер -> клиент. Автоматическое переподключение, простая семантика событий.

```
In [ ]: from fastapi import FastAPI
from fastapi.responses import StreamingResponse
import json

app = FastAPI()

async def progress_streamer(task_id: str):
    for progress in range(0, 101, 10):
        await asyncio.sleep(1)
        yield f"data: {json.dumps({'task_id': task_id, 'progress': progress})}\n\n"
        yield f"data: {json.dumps({'task_id': task_id, 'status': 'done'})}\n\n"

@app.get("/progress/{task_id}")
async def progress(task_id: str):
    return StreamingResponse(
        progress_streamer(task_id),
        media_type="text/event-stream",
    )
```

Формат SSE:

- Каждое событие начинается с `data:` и заканчивается двумя `\n\n`.
- Можно добавлять `id:`, `event:`, `retry:`.

Когда использовать SSE вместо WebSocket:

- Односторонний поток (сервер пушит прогресс, логи, метрики).
- Не нужен дуплекс.
- Работает через обычные HTTP-прокси и балансировщики.

9.4.3. WebSocket для real-time inference

WebSocket идеален для сценариев, где клиент и сервер обмениваются данными в реальном времени:

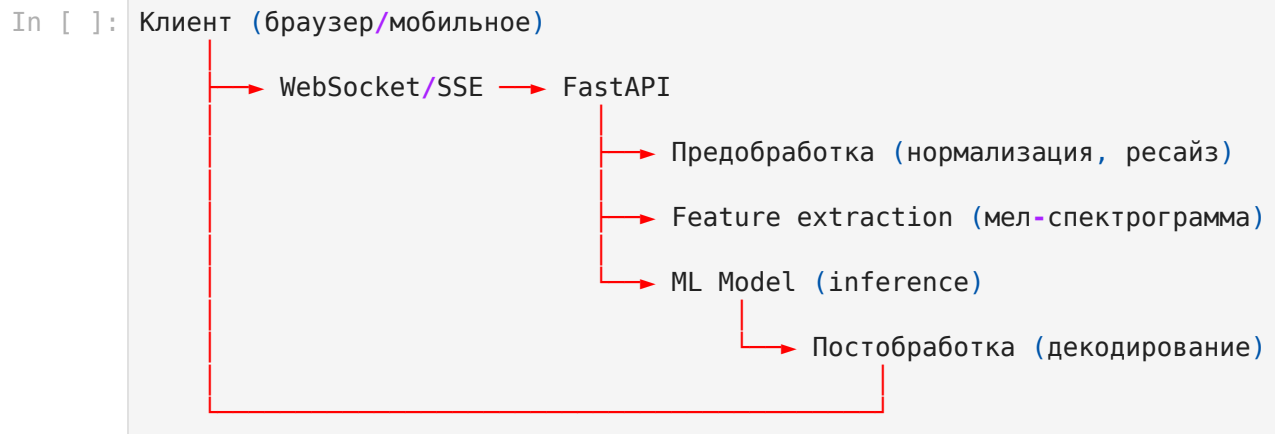
- **Распознавание речи:** клиент отправляет аудио-чанки, сервер возвращает промежуточные транскрипции.
- **Видеоаналитика:** клиент стримит кадры, сервер возвращает bounding boxes.
- **Интерактивные чат-боты:** клиент отправляет сообщения, сервер стримит ответ токенами.

```
In [ ]: from fastapi import FastAPI, WebSocket

app = FastAPI()

@app.websocket("/ws/transcribe")
async def transcribe(websocket: WebSocket):
    await websocket.accept()
    try:
        while True:
            audio_chunk = await websocket.receive_bytes()
            text = await speech_model.transcribe_chunk(audio_chunk)
            await websocket.send_text(text)
    except Exception:
        await websocket.close()
```

Архитектура real-time ML:



Latency budget для real-time:

Компонент	Бюджет
Сеть (клиент -> сервер)	20-50 мс
Десериализация	1-5 мс
Предобработка	5-20 мс

Компонент	Бюджет
Inference (GPU)	30–100 мс
Постобработка	1–5 мс
Сериализация + сеть обратно	10–30 мс
Итого	70–210 мс

Для интерактивного опыта (голосовой помощник) latency должна быть < 300 мс. Это накладывает жёсткие ограничения на размер модели и сложность предобработки.

9.5. Feature Store и управление данными

9.5.1. Offline vs Online фичи

Фичи (features) — входные переменные модели. В ML-сервисе они могут приходить из разных источников:

Тип	Примеры	Источник	Latency требования
Online фичи	Текущая геолокация, время суток, последний клик	Запрос, Kafka, Redis	< 10 мс
Offline фичи	История покупок за год, агрегаты по пользователю	Data Warehouse, S3	Предвычислены
Near-real-time	Счётчик просмотров за последний час	Flink, Kafka Streams	< 100 мс

Проблема: если для каждого предсказания нужно выполнять SQL-запрос к Data Warehouse на 100 ГБ данных — latency будет секундами. Фичи должны быть **предвычислены** и храниться в low-latency хранилище.

9.5.2. Feast, Tecton: концепция Feature Store

Feature Store — это управляемый слой между хранилищами данных и ML-сервисом. Он обеспечивает:

- **Единый источник правды:** одни и те же фичи используются при обучении (offline) и инференсе (online).
- **Версионирование:** фичи версионировются вместе с моделью.
- **Мониторинг:** отслеживание дрейфа фичей (feature drift).

- **Low-latency serving:** online фичи хранятся в Redis/DynamoDB.

Feast (open-source):

```
In [ ]: from feast import FeatureStore

store = FeatureStore(repo_path=".")

# Получение online фичей для конкретных сущностей
features = store.get_online_features(
    features=[
        "user_features:age",
        "user_features:purchase_count_7d",
        "product_features:category"
    ],
    entity_rows=[{"user_id": "123", "product_id": "456"}]
).to_dict()
```

Tecton (managed, enterprise): полностью управляемый feature store с автоматической материализацией, мониторингом и governance.

9.5.3. Кэширование фичей в Redis для low-latency inference

Для простых сценариев (без полноценного Feature Store) можно использовать Redis как кэш предвычисленных фичей:

```
In [ ]: import json
import hashlib

async def get_features(user_id: str, redis: Redis) -> dict:
    # Пробуем кэш
    cached = await redis.get(f"features:{user_id}")
    if cached:
        return json.loads(cached)

    # Иначе вычисляем (или загружаем из DWH)
    features = await compute_user_features(user_id)

    # Кладём в кэш с TTL
    await redis.setex(
        f"features:{user_id}",
        300, # 5 минут
        json.dumps(features)
    )
    return features

@app.post("/predict")
async def predict(input_data: InputModel, redis: Redis = Depends(get_redis)):
    features = await get_features(input_data.user_id, redis)
```



```
features.update(input_data.realtime_features)

result = await model_service.predict(features)
return {"prediction": result}
```

Кэширование согласованности (cache consistency):

- **Cache-aside:** приложение само управляет кэшем (читает, пишет, инвалидирует).
- **Write-through:** данные пишутся одновременно в БД и кэш.
- **Read-through:** кэш сам загружает данные из БД при промахе.

Для ML-фичей обычно используется **cache-aside** с коротким TTL: фичи не критичны для консистентности (модель устойчива к небольшому устареванию), но критичны для latency.

9.5.4. Математическая подоплека: дрейф фичей и распределения

Модель обучена на распределении $P_{\text{train}}(X)$. На инференсе распределение может измениться — **covariate shift**:

$$P_{\text{train}}(X) \neq P_{\text{inference}}(X)$$

Если фичи в кэше устарели, они представляют $P_{\text{stale}}(X)$, отличное от актуального. Это деградирует качество модели.

Мониторинг: отслеживать статистики фичей (среднее, стандартное отклонение, перцентили) в реальном времени и сравнивать с обучающей выборкой. Если расстояние (например, Wasserstein distance) между распределениями превышает порог — тревога.

$$W(P_{\text{train}}, P_{\text{inference}}) = \inf_{\gamma \in \Gamma} \mathbb{E}_{(x,y) \sim \gamma} \|x - y\|$$

где Γ — множество всех совместных распределений с маргиналями P_{train} и $P_{\text{inference}}$.

Итог модуля 9

Концепция	Суть	Практическое применение
Синхронный inference	В рамках одного HTTP-запроса	Лёгкие модели < 100 мс, CPU
Асинхронный	Очередь + job ID + опрос	Тяжёлые модели,

Концепция	Суть	Практическое применение
inference	статуса	непредсказуемое время
Batch inference	Накопление запросов в батч	GPU-эффективность, динамический батчинг
ONNX Runtime	Универсальный оптимизированный движок	Ускорение 5–20× по сравнению с чистым PyTorch
Triton	Промышленный ML-сервер NVIDIA	Dynamic batching, ensemble, GPU sharing
BackgroundTasks	Фоновые операции в том же процессе	Логирование, уведомления (без гарантий доставки)
Celery	Распределённые задачи с persistence	Надёжные фоновые задачи, retry, масштабирование
arq	Async-native очередь на Redis	Современный async стек без legacy
StreamingResponse	Потоковая передача чанков	Token-by-token генерация текста
SSE	Односторонний push поверх HTTP	Прогресс-бары, логи, метрики
WebSocket	Дуплекс real-time	Распознавание речи, видеоаналитика
Feature Store	Управление online/offline фичами	Единый источник, low-latency serving
Redis кэш	Предвычисленные фичи	Снижение latency с DWH-секунд до миллисекунд

В **Модуле 10** мы перейдём к продакшну: контейнеризация, конфигурация, observability (логи, метрики, трассировка), масштабирование и CI/CD.